Section 14.3: Media & Content Platforms

> *Part 3 of 8 in the System Design Interview Series*

Executive Overview

Media platforms handle large binary objects, metadata, and user-generated content at planetary scale. The key interview differentiators are: understanding the transcoding pipeline, CDN tiering strategy, and adaptive bitrate mechanics. Storage cost at scale is always a constraint — demonstrate awareness of tiering and deduplication.

---

1. Design a Video Streaming Platform (YouTube / Netflix)

Requirements Analysis

Dimension	Requirement
Functional	Video upload, transcoding, streaming, metadata search
Scale	1B videos, 10M concurrent streams
Startup Latency	< 2 seconds to first frame
Quality	Adaptive bitrate, global delivery

Scale reality check: YouTube ingests ~500 hours of video per minute. Your architecture must handle variable-rate ingestion bursts, not just steady-state.

Upload & Transcoding Pipeline

```
Client → Ingest Service (validation, chunking) → Message Queue (Kafka)
                                                          ↓
                                                 Transcoding Workers (pool)
                                                          ↓
                                  [1080p/5Mbps] [720p/3Mbps] [480p/1.5Mbps] [360p/750Kbps]
                                                          ↓
                                                 Packaging (HLS / DASH segments)
                                                          ↓
```

```
                                                CDN Origin Storage (S3)
                                                        ↓
                                                Metadata Catalog (update status)
```

Why a queue between ingest and transcoding? Transcoding is CPU-intensive and slow (a 1-hour video takes minutes to transcode). The queue decouples ingestion (fast, user-facing) from processing (slow, background). During upload spikes, the queue absorbs the burst.

Transcoding Stages in Detail

1. **Validation**: Verify file integrity, check format/codec support, reject corrupt uploads early
2. **Demux**: Separate audio and video elementary streams
3. **Decode**: Decompress to raw frames (ffmpeg)
4. **Transcode**: Re-encode at multiple bitrate/resolution "ladder" rungs
5. **Mux**: Package back into HLS/DASH container with segment boundaries at keyframes
6. **Thumbnail extraction**: Extract frames at 10% intervals for preview scrubbing
7. **Audio**: Normalise loudness, generate separate audio tracks for multi-language

Adaptive Bitrate Streaming (ABR)

HLS manifest tells the player what streams are available:

```
#EXTM3U
#EXT-X-STREAM-INF:BANDWIDTH=5000000,RESOLUTION=1920x1080
1080p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=3000000,RESOLUTION=1280x720
720p/playlist.m3u8
#EXT-X-STREAM-INF:BANDWIDTH=750000,RESOLUTION=640x360
360p/playlist.m3u8
```

ABR algorithm comparison:

Algorithm	Behaviour	Best For
Throughput-based	Estimates bandwidth from last segment download speed; selects matching bitrate	Simple, fast adaptation
Buffer-based (BBA)	Maintains buffer target; upgrades only when buffer is healthy	Stability — fewer quality switches
Hybrid (BOLA, Pensieve)	Combines bandwidth estimate + buffer occupancy + QoE model	Production default (YouTube, Netflix)

Startup optimisation: Always start at a low bitrate (360p) for the first segment regardless of estimated bandwidth. This gets the first frame on screen fast. Ramp up after the buffer is established.

CDN Strategy

Strategy	How It Works	Storage Cost	Latency	Best For
Push CDN	Videos proactively pushed to all edge nodes on upload	High (all content at all edges)	Very low	Popular content, small library
Pull CDN	Edge fetches from origin on first request, caches	Low (only popular content cached)	First request slower	Long-tail content
Hybrid (Tiered)	Hot content pushed; tail content pulled	Medium	Low for hot, medium for tail	General platform (YouTube/Netflix)

Multi-tier cache hierarchy:

```
Edge PoP (10TB SSD) → Regional Cache (100TB HDD) → Origin (S3, petabyte scale)
    ↓                          ↓                              ↓
Response < 5ms          Response < 20ms              Response < 200ms
```

Cache hit ratio target: > 95% at edge. Content that misses edge but hits regional avoids expensive origin egress.

Interview Problem 1 — Minimising Startup Latency

"Design a video streaming service that achieves < 1 second startup time while keeping storage costs reasonable. What are the key trade-offs?"

Solution framework:

1. **Low bitrate first segment**: Pre-generate a 360p segment 0 (typically 2–4 seconds of video) and cache it at every edge PoP unconditionally. This segment is small (~750KB) and always warm.
2. **Variable GOP (Group of Pictures)**: Set shorter keyframe intervals for the first 30 seconds of each video (every 1 second vs every 4 seconds). This reduces segment duration and allows the player to start faster.
3. **Pre-warming popular content**: Use view velocity signals (first 15 minutes after publish) to predict whether a video will be popular. Pre-push to edge CDNs proactively.

4. **Storage cost control**: 1080p is ~5GB/hour. 720p is ~3GB/hour. Store 1080p only on origin and regional caches; push 720p and below to edge. 40% storage saving at the edge.
5. **Perceptual trick**: Show the first frame as a static image while buffering begins. Users perceive startup as instant even if video playback starts 200ms later.

Interview Problem 2 — Handling a Viral Upload

"A video goes viral immediately after upload, before transcoding is complete. 10M users simultaneously try to watch it. How do you handle this?"

Solution framework:

1. **Progressive availability**: Make the original (non-transcoded) version available immediately at one quality level, if the codec is browser-compatible (H.264). Serve this while transcoding runs in background.
2. **Priority queue**: Promote the viral video's transcoding job to high priority. Scale up the transcoding worker pool horizontally (Kubernetes HPA based on queue depth).
3. **Request coalescing at CDN**: The first request to each CDN edge node triggers an origin fetch. Subsequent concurrent requests for the same segment are held and served from the same origin response (stampede protection).
4. **Graceful degradation**: If only 360p is ready, serve it. Update the HLS manifest incrementally as higher resolutions complete transcoding. Players automatically upgrade to higher bitrate on manifest refresh.
5. **Notify users**: If nothing is available yet, show a "Processing" state in the UI rather than a buffering spinner.

---

2. Design Cloud File Storage (Google Drive / Dropbox / S3)

Requirements Analysis

Dimension	Requirement
Functional	File CRUD, sync across devices, sharing with permissions
Scale	1B users, 100PB stored
Durability	11 nines (99.999999999%)
Performance	

Dimension	Requirement
	< 100ms for metadata operations; streaming download

Core Architecture

```
Client App → Metadata Service (PostgreSQL) → tracks file structure, versions
           → Chunk Service   (S3-backed)   → stores actual bytes
           → CDN / Transfer Accel          → fast download delivery
```

Why separate metadata from content? Metadata operations (rename, share, list folder) are frequent and small — they need ACID and < 100ms. Content is large, infrequent, and can tolerate higher latency. Mixing them would require all DB queries to scan potentially petabytes.

Chunking Strategy

```
1GB video.mp4 → split into 8MB chunks → [chunk_0][chunk_1]...[chunk_127]

Benefits:
- Parallel upload: All 128 chunks upload simultaneously (8× faster on 8-core)
- Resumable upload: Track which chunks uploaded; resume from last successful
- Deduplication: Identical content (e.g., same logo in two files) stored once
- Delta sync: Only changed chunks re-upload on file modification
```

Chunk size trade-off: - Larger chunks (32MB): Fewer metadata records, better for large files - Smaller chunks (1MB): Better deduplication granularity, faster resumability for slow connections - Production default: 8MB (Dropbox) or variable (AWS S3 multipart)

Data Model

```
CREATE TABLE files (
    file_id     UUID        PRIMARY KEY,
    owner_id    BIGINT      NOT NULL,
    name        TEXT        NOT NULL,
    size_bytes  BIGINT,
    chunk_refs  JSONB,      -- [{chunk_id, offset, size}]
    version     INT         DEFAULT 1,
    created_at  TIMESTAMP,
    updated_at  TIMESTAMP
);

CREATE TABLE chunks (
    chunk_id      UUID    PRIMARY KEY,
    storage_key   TEXT,   -- S3 object key
    content_hash  TEXT,   -- SHA-256 for dedup
```

```
    size_bytes    INT,
    ref_count     INT     DEFAULT 1  -- for garbage collection
);
```

Deduplication

Content-addressed storage: hash each chunk → only store unique hashes.

```
User A uploads photo.jpg (hash: abc123)
User B uploads same photo.jpg → hash: abc123 → already stored!
→ Just create a new file record pointing to existing chunk
→ No duplicate bytes stored
```

Deduplication savings: Enterprise storage can achieve 60–80% space reduction through deduplication of common files (OS images, shared assets).

Delta Sync (How Dropbox Works)

```
File changes on client:
1. Compute block-level rolling hash of new file (rsync algorithm)
2. Compare against stored chunk hashes for this file
3. Identify changed/new blocks (typically a small fraction)
4. Upload only the changed blocks
5. Update file record's chunk_refs in metadata service
```

Example: Edit a 100MB PowerPoint to change one slide → maybe 2–3 chunks change (2MB). Upload is 2MB not 100MB.

Trade-Off Matrix

Feature	Performance Impact	Storage Impact	Complexity
Chunking	Parallel upload — high perf gain	+5–10% metadata overhead	Medium
Deduplication	Slight write overhead (hash compute)	40–80% storage saving	High
Delta sync	Dramatic bandwidth reduction	Minimal	High
Versioning	+1 file record per version	Proportional to version count	Low

---

Interview Problem 3 — Sync Conflict Resolution

"A user edits a file on two devices simultaneously while offline. When both devices reconnect, how do you resolve the conflict?"

Solution framework:

1. **Conflict detection**: Compare `(device_id, version_at_last_sync)` when uploading. If the server version is higher than the client's base version, a conflict exists.
2. **Automatic resolution** (for binary files like .docx): Cannot safely auto-merge. Create two versions: `Report (Macbook's conflicted copy).docx` and keep original as `Report.docx`. Both visible to user.
3. **Automatic resolution** (for text files): Attempt a 3-way merge (common ancestor + both branches). Show conflicts inline (like `git merge`). If clean, commit merged version automatically.
4. **Client responsibility**: The client that uploads second detects the conflict by receiving a 409 Conflict response and must decide how to present it to the user.
5. **UX principle**: Never silently lose data. Always surface conflicts to the user rather than picking one version arbitrarily.

Section Summary: Media Platform Decision Points

Design Choice	Option A	Option B	When to Choose A	When to Choose B
Transcoding pipeline	Synchronous (inline)	Async queue-based	< 1min videos	Long-form / bursty uploads
CDN strategy	Push (proactive)	Pull (lazy)	Small library, viral content	Long-tail, cost-sensitive
ABR algorithm	Throughput-based	Buffer-based	Fast adaptation needed	Stability prioritised
File chunking	1–4MB	8–32MB	Mobile / slow connections	Large files, fast links
Conflict resolution	Auto-merge (text)	Fork + user decides (binary)	Text/code files	Binary documents